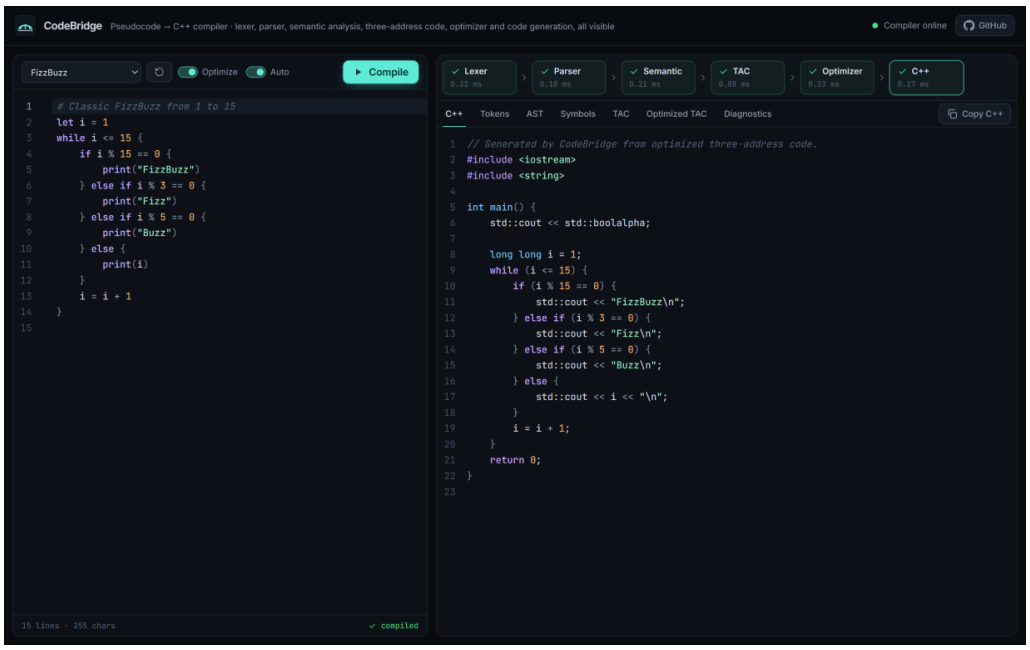
COMPILER DESIGN MINI-PROJECT REPORT

CodeBridge

A Pseudocode-to-C++ Compiler with a Phase-by-Phase Web Workbench

Lexer · Recursive-descent parser · Semantic analysis · Three-address code · Optimizer · C++17 code generation



Submitted by: [Student Name]
Roll number: [Roll Number]
Department: [Department]
Institution: [Institution]
Project guide: [Guide Name]
Academic year: [Academic Year]

Live application: https://codebridge-swart.vercel.app
Source code: https://github.com/ashmita-dev/CodeBridge

Abstract

CodeBridge is a compiler that translates a small, statically typed pseudocode language into C++17 and exposes every compiler phase in a web workbench. The compiler is written by hand in Python, without lexer or parser generators. It consists of a scanner, a recursive-descent parser that builds a typed abstract syntax tree (AST), a semantic analyser with a scoped symbol table and type checker, a three-address code (TAC) generator, an optimizer that applies eleven classical passes until a fixed point, and a code generator that rebuilds structured C++ from the optimized TAC. A FastAPI service exposes the pipeline through `POST /compile`, and a React and CodeMirror front end shows the tokens, AST, symbol table, original and optimized TAC, generated C++ and structured diagnostics for the program being edited. The system is deployed with the front end on Vercel and the API on Render. Correctness is checked by 213 automated backend tests and 20 frontend tests. The generated C++ of every example program is compiled with g++ and its output compared with an independent reference interpreter, and the optimizer is checked against 300 randomly generated programs.

Keywords: compiler design, lexical analysis, recursive-descent parsing, abstract syntax tree, symbol table, type checking, three-address code, code optimization, source-to-source translation, C++, FastAPI, React.

1. Introduction

A compiler course usually teaches lexical analysis, parsing, semantic analysis, intermediate code and optimization as separate topics, often on paper. Students rarely see one program travel through a complete, working compiler or observe how an optimizer rewrites intermediate code. CodeBridge was built to close that gap: it is a complete source-to-source compiler whose every intermediate artifact is visible and interactive in the browser, while the output is a real, compilable C++ program.

1.1 Problem statement

Design and implement a compiler that accepts programs in a simple pseudocode language, reports lexical, syntax and semantic errors precisely, generates and optimizes intermediate code, and emits equivalent C++ source. The result of every phase must be presented to the user through a web interface, and the translation must be demonstrably correct: the C++ program must print exactly what the source program means.

1.2 Motivation

- **Learning by observation:** seeing tokens, the AST, the symbol table and TAC for one's own program makes the theory concrete.
- **Correctness as a requirement:** a translator that produces plausible but wrong C++ is worse than none, so the project emphasizes verified semantics over feature count.
- **Accessibility:** a hosted web application needs no local toolchain, so anyone can try the compiler from a browser.

2. Objectives and Scope

2.1 Objectives

1. Design a small but complete language: variables, the types int, float, string and bool, arithmetic, comparisons, logical operators, `print`, `if`/`else if`/`else` and `while`.
2. Implement every compiler phase by hand and keep each phase testable without the web framework.
3. Report errors as structured diagnostics with a stage, a code, a message and an exact line and column.
4. Generate three-address code, optimize it safely, and generate C++17 that behaves exactly like the source.

5. Present all phases in a responsive web workbench and deploy it publicly.

2.2 Scope

In scope: the language above; all six compiler phases; a JSON API; the web workbench; automated and live verification. **Out of scope by design:** functions, arrays, `for` loops, `break`/`continue` and input; and executing user programs on the server. CodeBridge generates C++ but never runs it in production, which keeps the hosted service free of arbitrary code execution.

3. Requirements

Table 1. Functional (F) and non-functional (N) requirements

ID	Requirement	How it is met
F1	Translate valid programs to compilable C++17	Six-phase pipeline; g++ build-and-run tests
F2	Show tokens, AST, symbols, TAC, optimized TAC and C++	API returns every artifact; one tab per artifact
F3	Report lexical, syntax and semantic errors with line:column	Structured diagnostics (codes L, P, S)
F4	No downstream artifacts after a failing phase	Pipeline stops; TAC and C++ are null on failure
F5	Optimize intermediate code without changing behaviour	Eleven passes; differential testing
N1	User input must never crash the server	Input limits, recursion limits, JSON 500 handler
N2	Never execute user-generated C++ on the backend	No execution endpoint exists
N3	Configuration through environment variables, no secrets	`CORS_ORIGINS`, `MAX_SOURCE_CHARS`, `VITE_API_URL`
N4	Responsive, accessible interface	Responsive layout down to phone widths (Figure 9)

4. Technology Stack

Table 2. Technology stack

Layer	Technology (version in the repository)	Role
Compiler and API	Python 3.13.15, FastAPI 0.143.0, Pydantic 2.14.1, Uvicorn 0.54.0	Compiler package, HTTP API, validation
Frontend	React 19.3, TypeScript 7.0, Vite 8.3, Tailwind CSS 4.3	Workbench UI and build
Editor	CodeMirror 6 (lang-cpp, lint, language)	Source editor, C++ viewer, error underlines
Testing	pytest 9.1, httpx2, Vitest 5.0, g++ 16.1 (MSYS2)	Unit, API, end-to-end and build-and-run tests
CI and hosting	GitHub Actions, Render (API), Vercel (frontend)	Automated tests; production deployment

5. System Architecture

CodeBridge is a two-tier web application (Figure 1). The browser runs a single-page React application served by Vercel. It sends the editor contents to the FastAPI backend on Render and renders the JSON response. The compiler lives in the package `app/compiler`, which has no web dependencies; `pipeline.compile_source()` runs the phases in order, records the time each one takes and stops at the first phase that reports errors.

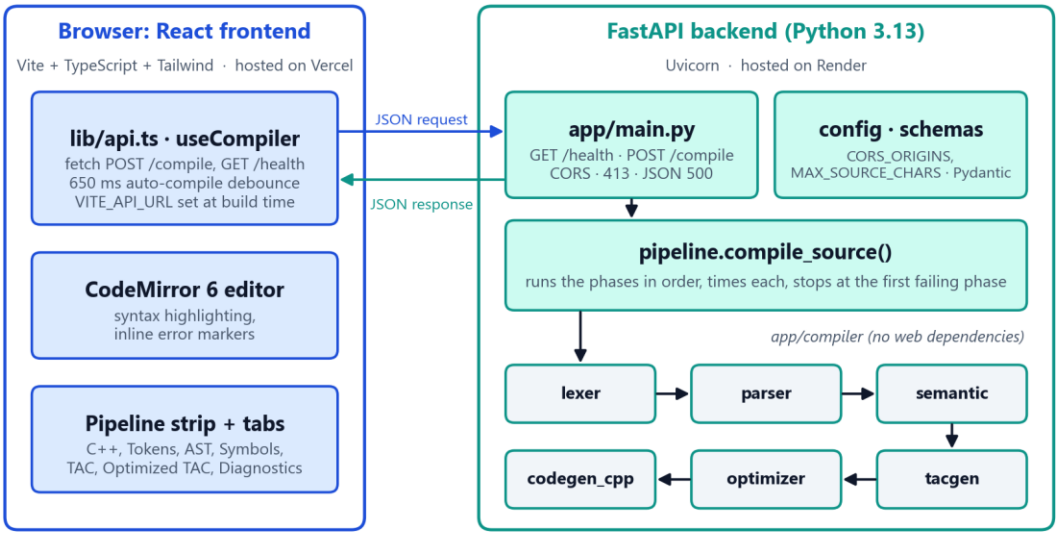


Figure 1. System architecture, drawn from the modules in frontend/src and backend/app.

5.1 API integration

The API has two endpoints. `GET /health` returns `{"status": "ok"}` and is used by Render's health check and by the frontend's online indicator. `POST /compile` accepts `{"source": ..., "optimize": true}` and returns `success`, the status of each phase (`ok`, `error` or `skipped`), the diagnostics, the tokens, the AST, the symbol table, the original and optimized TAC, the C++ source, optimization statistics and per-phase timings. Requests over `MAX_SOURCE_CHARS` (20,000) are rejected with HTTP 413, malformed bodies with 422, and an unexpected exception produces a JSON 500 response instead of a crash. CORS admits only the origins listed in `CORS_ORIGINS`; in production that is exactly `https://codebridge-swart.vercel.app`. The frontend's API base URL is inlined at build time from `VITE_API_URL`. In auto mode it recompiles 650 ms after the last keystroke, and on a slow first request it tells the user the free-tier server is waking up.

5.2 Deployment

Table 3. Production deployment

Component	Platform and configuration
Backend API	Render web service from the `render.yaml` Blueprint: root `backend`, `pip install -r requirements.txt`, `uvicorn app.main:app --host 0.0.0.0 --port $PORT`, health check `/health`, `PYTHON_VERSION=3.13.15`, `CORS_ORIGINS` set to the Vercel origin. URL: https://codebridge-api.onrender.com
Frontend	Vercel project built by `vercel.json` (`npm ci` and `npm run build` in `frontend/`, output `frontend/dist`), `VITE_API_URL=https://codebridge-api.onrender.com`. URL: https://codebridge-swart.vercel.app
Continuous delivery	Both platforms redeploy on every push to `main`; GitHub Actions runs all tests on each push.

6. Methodology

The compiler follows the classical analysis–synthesis structure (Figure 2). Each phase is a separate module with its own tests, and each consumes only the output of the previous phase. Development was incremental and test-driven: a phase was added only when the previous one was complete and tested, in the order language specification, lexer, parser, semantic analysis, TAC, optimizer and code generator, then API, frontend and deployment. To check meaning rather than just form, the test suite contains two **reference interpreters**, one for the AST and one for TAC. They are used as oracles for the optimizer and the code generator.

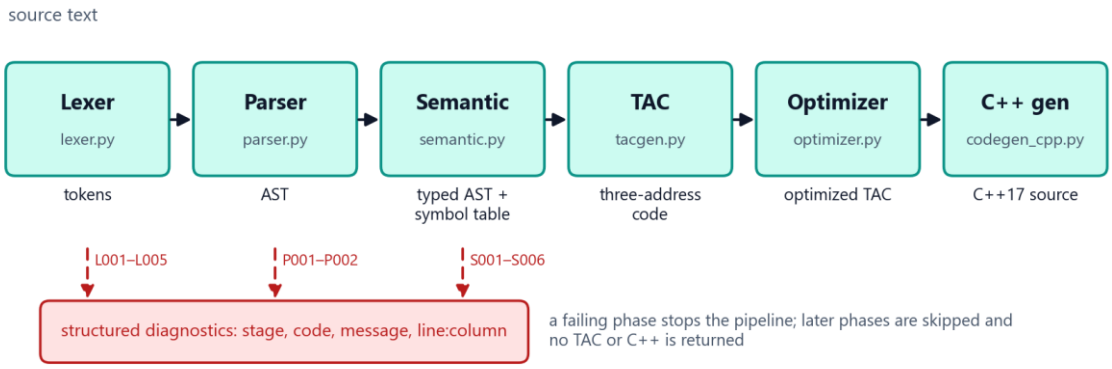


Figure 2. The six-phase compiler pipeline and its error paths.

7. Compiler Design

7.1 Source language

Statements end with a newline or `;`, blocks use braces and # starts a comment. A declaration is `let x = expr` or `let x: type = expr`; the only implicit conversion is int to float. `print` takes one or more comma-separated expressions, with or without parentheses (`print(a, b)` or `print a, b`). Semantics follow C++ deliberately: integers are 64-bit, `/` truncates toward zero, `%` takes the sign of the dividend, and/or short-circuit and floats print like `std::cout`. The grammar in EBNF (abridged from `parser.py`):

```
statement  = let_stmt | assign_stmt | print_stmt | if_stmt | while_stmt ;
print_stmt = "print" "(" [ args ] ")" | "print" args ;
if_stmt    = "if" expression block [ "else" ( if_stmt | block ) ] ;
expression = or_expr ;   or_expr = and_expr { "or" and_expr } ;   ...
term       = unary { ( "*" | "/" | "%" ) unary } ;   unary = "-" unary | primary ;
```

7.2 Lexical analysis

`lexer.py` is a hand-written scanner over the character stream. Operators are matched longest-first (<= before <), every token carries a 1-based span, and newlines become `NEWLINE` tokens except inside parentheses, so expressions may span lines. The lexer records an error and continues, so one pass reports every lexical error: invalid characters with hints such as "use `not`" for `!` (L001), malformed numbers such as `1.2.3` (L002), literals outside the 64-bit or finite float range (L003), unterminated strings (L004) and unknown escape sequences (L005).

7.3 Syntax analysis and the AST

`parser.py` is a predictive recursive-descent parser with one method per non-terminal. Operator precedence comes from the chain of grammar levels (`or` < `and` < `not` < equality < comparison < additive < multiplicative < unary), and left associativity from iteration inside each level. The parser builds dataclass nodes (`ast_nodes.py`) that carry source spans and serialize to JSON for the AST view. It stops at the first syntax error with a precise message, for example "expected ')' to close '(' but found '{'" (P001). Nesting depth (50) and expression height (150) are bounded (P002), so hostile input cannot overflow the Python stack.

7.4 Semantic analysis and the symbol table

`semantic.py` walks the AST with a symbol table implemented as a stack of scopes, one per block. It resolves names (undefined or undeclared variables, S001; redeclaration in one scope, S002), infers the type of every expression and checks initializers and assignments (S003), operand types (S004), `bool` conditions (S005) and division by a literal zero (S006). All semantic errors are collected; a variable with an invalid initializer is marked so its later uses do not cause cascading errors. Shadowed variables receive unique names (x, x_2) that also avoid C++ keywords and the TAC temporaries `tN`. Unused variables produce warning S100.

7.5 Intermediate code: three-address code

`tacgen.py` lowers the typed AST into typed TAC instructions: copies, binary and unary operations (including an explicit `(float)` conversion), labels, `goto`, `ifFalse`/`ifTrue` conditional jumps and `print`. Control flow uses the standard schemes, and `and`/`or` are compiled with jumps so they short-circuit; for example `d != 0 and 10 / d > 1` never divides by zero.

Table 4. Three-address code instruction forms (tac.py)

Instruction	Example	Meaning
Copy	`x = t1`	assign a value or temporary to a variable
Binary / unary	`t1 = a * b`, `t2 = -a`, `t3 = not c`	one operator, at most two operands
Conversion	`t4 = (float) i`	explicit int-to-float widening
Jumps	`goto L1`, `ifFalse t1 goto L2`, `ifTrue t1 goto L3`	unconditional and conditional control flow
Label / output	`L1:`, `print a, b`	jump target; print values separated by spaces

7.6 Optimization

`optimizer.py` repeats its passes until nothing changes. Constant and copy propagation, constant folding and algebraic simplification are local to basic blocks: known values are discarded at every label, where control paths merge, and whenever a variable they depend on is reassigned. Folding uses C++ semantics and refuses overflow, division by zero and non-finite results; floating-point identities are applied only when exact under IEEE-754. The other passes are self-assignment removal, branch folding, temporary coalescing, dead-temporary elimination, unreachable-code removal on the control-flow graph, and redundant-jump and unused-label removal. User variables and `print` statements are never removed. The example below is the exact compiler output for `let x = 2 * 3 + 4` followed by a loop; on the bundled optimizer showcase program the instruction count falls from 25 to 14 (−44%, Figure 6).

Original TAC

```
    t1 = 2 * 3
    t2 = t1 + 4
    x = t2
    i = 0
L1:
    t3 = i < x
    ifFalse t3 goto L2
```

Optimized TAC

```
    x = 10
    i = 0
L1:
    t3 = i < x
    ifFalse t3 goto L2
```

7.7 C++ code generation

`codegen_cpp.py` translates the optimized TAC, never the AST, so the C++ reflects the optimizations. It works in three steps: temporaries used once are folded back into expressions; jump patterns are recognized and rebuilt as `while`, `if/else` and `else if` (with a `goto` fallback that keeps the output correct if a pattern does not match); and the program is emitted with int as `long long`, float as `double`, string as `std::string`, parentheses chosen by precedence, escaped string literals and `std::cout` with `std::boolalpha`. The pair below is unedited compiler output; built with g++, the C++ prints `1 is odd`, `3 is odd`, `5 is odd` and `sum of even squares: 20`, the same as the reference interpreter.

CodeBridge source

```
let n = 1
let total = 0
while n <= 5 {
    if n % 2 == 0 {
        total = total + n * n
    } else {
        print n, "is odd"
    }
    n = n + 1
}
print("sum of even squares:", total)
```

Generated C++ (body of main)

```
long long n = 1;
long long total = 0;
while (n <= 5) {
    if (n % 2 == 0) {
        total = total + n * n;
    } else {
        std::cout << n << " is odd\n";
    }
    n = n + 1;
}
std::cout << "sum of even squares: " << total << "\n";
```

7.8 Error handling

Every diagnostic has a stage (lexical, syntax, semantic or internal), a severity, a code, a message, a start and end position and optional hints. The editor underlines each error at its span, and clicking a diagnostic moves the cursor to it (Figure 8). When any phase fails, later phases are reported as skipped and the response contains no TAC and no C++, so the interface never shows stale or invented output.

8. Implementation and Screenshots

The compiler package is about 2,600 lines of Python in fourteen files, the FastAPI layer about 150 lines and the backend tests about 1,500 lines. The frontend is about 1,900 lines of TypeScript and TSX: a header with the online indicator, a pipeline strip with per-phase status and timing, the editor panel with an example selector, reset, optimize and auto-compile toggles, and an output panel with seven tabs and copy buttons. Deep links such as `?example=fizzbuzz&tab=cpp` open a given example and tab. All screenshots below were captured from the live deployment at https://codebridge-swart.vercel.app on 13 October 2026.

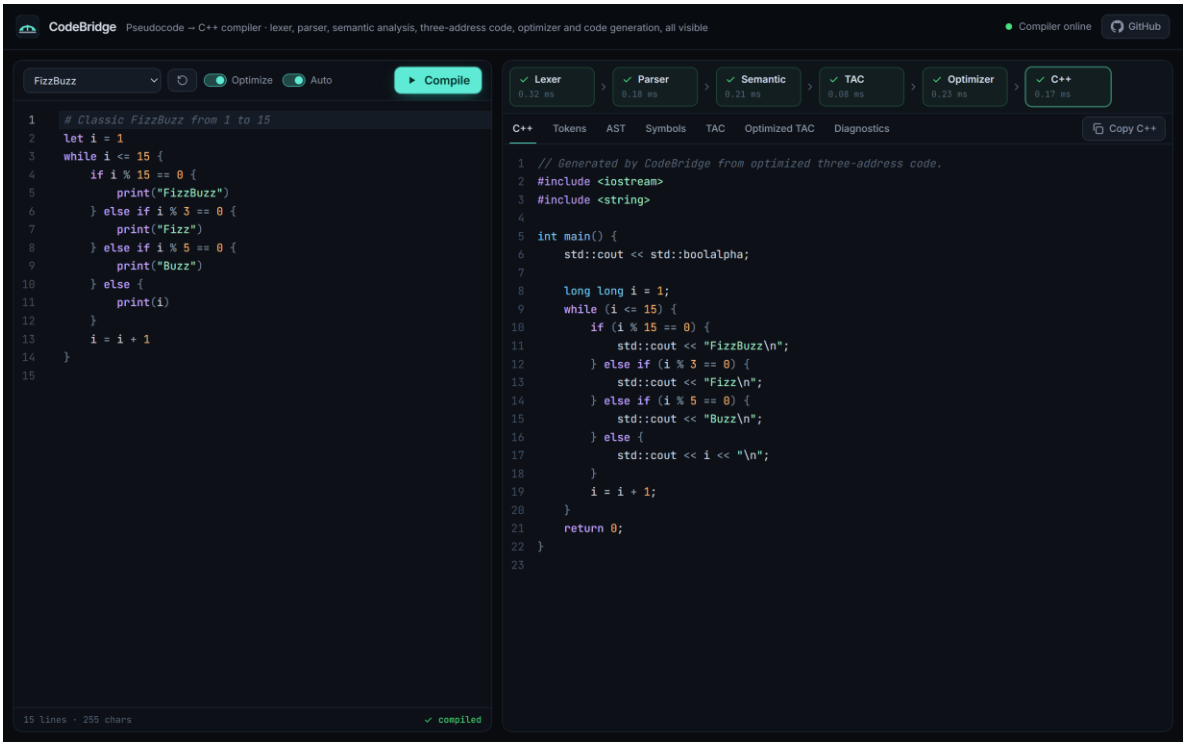


Figure 3. Live workbench: FizzBuzz source (left) and the generated C++ with a recovered else-if chain (right).

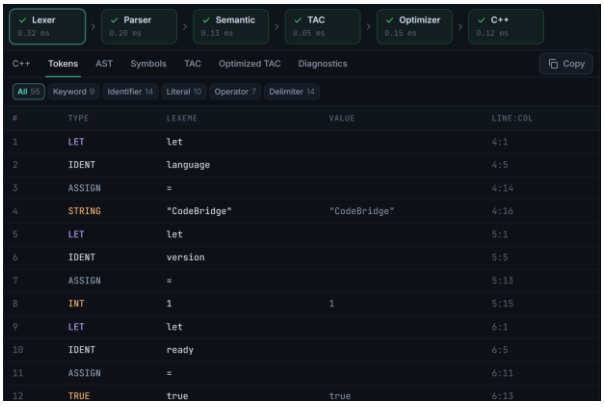


Figure 4. Tokens tab: type, lexeme, value and line:column, with category filters (cropped).

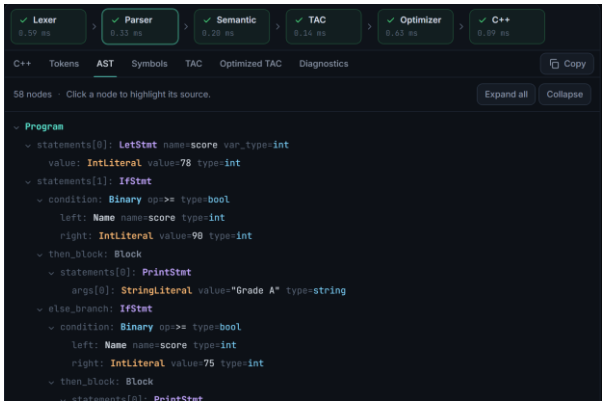


Figure 5. AST tab: typed nodes of an else-if chain; clicking a node selects its source (cropped).

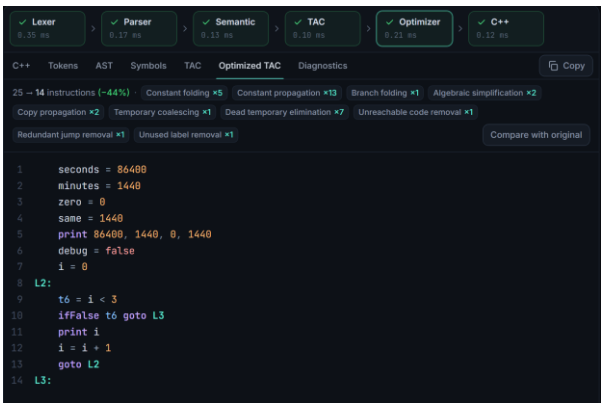


Figure 6. Optimized TAC with per-pass statistics: 25 → 14 instructions (cropped).

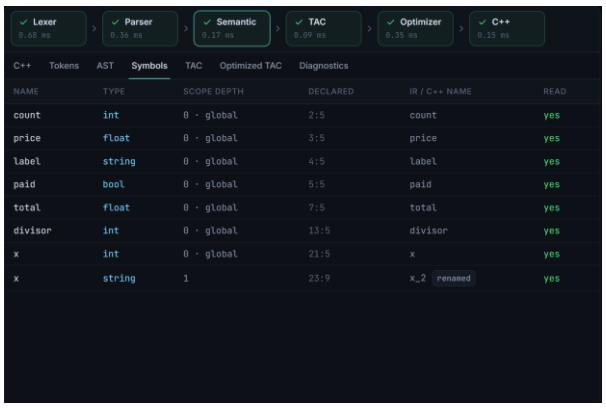


Figure 7. Symbol table: types, scope depth and the renamed shadowed variable x_2 (cropped).

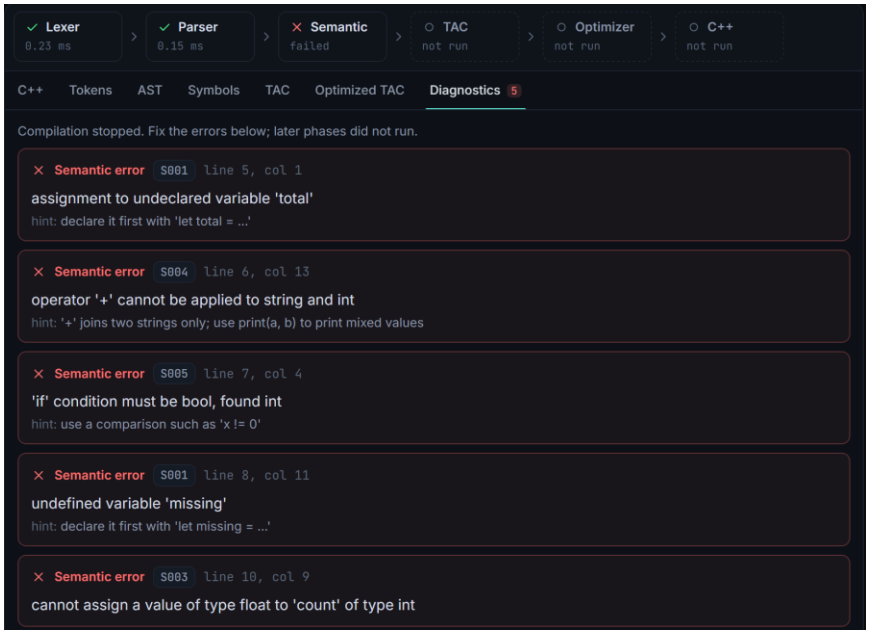


Figure 8. Diagnostics tab: five semantic errors with codes, line:column positions and hints; later phases not run (cropped).



Figure 9. Phone-width layout, 500 px viewport (cropped).

9. Testing Methodology

Testing combines automated suites, run locally and by GitHub Actions on every push, with manual and scripted checks of the deployed system. Each example program is checked four ways and the outputs must be identical: the AST interpreter, a TAC interpreter on the original TAC, the same interpreter on the optimized TAC, and a native binary built from the generated C++ with g++ (`-std=c++17 -Wall -Wextra`). The optimizer is additionally checked against 300 random programs, and nine programs with hand-derived expected output guard against a mistake shared by the interpreter and the compiler.

Table 5. Automated test suites (213 backend + 20 frontend)

Suite (file)	Tests	What it covers
Lexer (`test_lexer.py`)	12	token kinds, longest match, positions, CRLF, every lexical error code
Parser and AST (`test_parser.py`)	48	15 precedence cases, statements, both print forms, 21 syntax-error cases
Semantic (`test_semantic.py`)	31	S001–S006 with exact positions, scopes, shadowing, no cascades
TAC (`test_tac.py`)	18	exact TAC shapes, short-circuit, TAC vs. AST interpreter
Optimizer (`test_optimizer.py`)	28	safe folding, no propagation into loops, 300 random programs
C++ generation (`test_codegen.py`)	36	structure, parentheses, escaping; g++ build and run of all programs

Suite (file)	Tests	What it covers
API and health (`test_api.py`, `test_health.py`)	34	payloads, 413/422/500, CORS, every example over HTTP
End-to-end (`test_e2e_server.py`)	6	live Uvicorn server → HTTP → g++ → output
Frontend (Vitest, 6 files)	20	API client, AST view, phases, examples, positions, output panel

10. Verified Results

10.1 Automated results

On the final commit all **213 backend tests passed**, including the g++ build-and-run tests and the end-to-end server test, and the frontend type check, **20 Vitest tests** and the production build passed. GitHub Actions CI (Ubuntu) passed on the same commits. Locally, a 19,860-character program compiled through all six phases in 0.057 s (median of five runs on the Windows 11 development machine).

Table 6 lists the twelve bundled example programs. Every valid example is compiled, built with g++ and run by the test suite, and its output must equal the stored expected output; every error example must fail in the intended phase with no TAC or C++. The TAC counts are the compiler's own optimization statistics. Programs whose branch conditions depend on constants shrink the most (`conditions`: 43 to 7), while loop-heavy programs shrink little because propagation stops at loop labels.

Table 6. Example programs and their verified results

Example	Exercises	TAC before → after	Verified result
hello	variables, strings, print	10 → 9	g++ output = expected
arithmetic	precedence, associativity, int/float division	36 → 14	g++ output = expected
conditions	else-if chains, nested branches, and/or/not	43 → 7	g++ output = expected
loops	sum, factorial, countdown	35 → 30	g++ output = expected
fizzbuzz	loop with an else-if chain	30 → 29	g++ output = expected
primes	nested loops, short-circuit conditions	33 → 30	g++ output = expected
fibonacci	block-scoped variables in a loop	15 → 13	g++ output = expected
types	annotations, widening, escapes, shadowing	31 → 14	g++ output = expected
optimization	folding, propagation, simplification	25 → 14	g++ output = expected
error_lexical	invalid characters, malformed numbers	not generated	3 errors (L001, L002)
error_syntax	unbalanced parenthesis	not generated	1 error (P001)
error_semantic	undefined variables, type mismatches	not generated	5 errors (S001, S003–S005)

10.2 Manual and scripted checks of the deployed system (13 October 2026)

Table 7. Checks performed manually or with one-off scripts (not part of the automated suite)

Check	Result
`GET https://codebridge-api.onrender.com/health`	HTTP 200, `{"status":"ok"}`
CORS preflight from the Vercel origin / from another origin	allowed / rejected with HTTP 400
Live app in headless Microsoft Edge	"Compiler online"; FizzBuzz and the optimizer showcase compiled through all six phases
C++ returned by the live API for fizzbuzz, primes and types, built with g++	output identical to `examples/expected`
Live syntax and semantic error requests	P001 and S001 diagnostics, no C++ returned
Warm `POST /compile` round trip from the development machine (5 requests)	0.55 s to 1.45 s
Pre-deployment audit: 21 hand-checked programs, 18 error cases, 200 random programs	all matched; 192 random programs matched g++, 8 skipped as integer-overflow (undefined behaviour)

11. Limitations and Future Scope

- **Language:** no functions, arrays, `for` loops, `break`/`continue` or input.
- **Error recovery:** the parser reports only the first syntax error (lexical and semantic errors are all reported).
- **Optimization scope:** propagation is local to basic blocks, so loop-invariant values are not propagated and there is no loop-invariant code motion.
- **Run-time behaviour:** division by a variable that is zero, or integer overflow, is undefined behaviour exactly as in C++; the generated program is not executed by the service.
- **Hosting:** the free Render instance sleeps when idle, so the first request can take up to about a minute.

Future scope: functions with a call stack in TAC; panic-mode error recovery; global data-flow analysis (reaching definitions, liveness) and SSA form; and running the generated C++ in the browser with WebAssembly, which would show program output without executing code on the server.

12. Conclusion

CodeBridge implements a complete compiler pipeline for a small language and makes each phase observable in a deployed web application. Hand-written lexing, recursive-descent parsing, scoped semantic analysis, TAC generation, safe optimization and structured C++ generation work together, and testing against reference interpreters and real g++ binaries, both automatically and on the live deployment, gives strong evidence that the generated C++ preserves the meaning of the source program.

References

[1] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson, 2006.

[2] K. D. Cooper and L. Torczon, Engineering a Compiler, 2nd ed. Morgan Kaufmann, 2011.

[3] R. Nystrom, Crafting Interpreters. Genever Benning, 2021.

[4] ISO/IEC 14882:2017, Programming languages: C++.

[5] FastAPI documentation, https://fastapi.tiangolo.com; Pydantic documentation, https://docs.pydantic.dev.

[6] React documentation, https://react.dev; Vite documentation, https://vite.dev; CodeMirror 6, https://codemirror.net.

[7] pytest documentation, https://docs.pytest.org; Vitest documentation, https://vitest.dev.

[8] Render documentation (Blueprints), https://render.com/docs; Vercel documentation, https://vercel.com/docs.